

КАФЕДРА Системы обработки информации и управления

Тема: «Реализация алгоритма Policy Iteration»

Студент	ИУСИ-22М		Тавуз Мохамад
	(группа)	(подпись, дата)	(И.О. Фамилия)
Преподаватель			Ю.Е.Гапанюк
		(подпись, дата)	(И.О. Фамилия)

1

1) Введение

Обучение с подкреплением является одним из направлений машинного обучения, в котором агент обучается принимать решения на основе взаимодействия с окружающей средой. На каждом шаге агент находится в некотором состоянии, выбирает действие, получает вознаграждение и переходит в новое состояние. Цель агента состоит в выборе такой стратегии поведения, которая позволяет максимизировать суммарное ожидаемое вознаграждение.

Цель данной лабораторной работы — изучение и программная реализация алгоритма Policy Iteration для дискретной среды обучения с подкреплением. Данный алгоритм относится к методам динамического программирования и используется для нахождения оптимальной стратегии в марковском процессе принятия решений при известной модели среды.

В ходе работы была выбрана среда Taxi-v3 из библиотеки Gym. В данной среде агент управляет такси, которое должно забрать пассажира в одной из заданных точек и доставить его в пункт назначения. Для решения задачи были реализованы этапы оценивания стратегии Policy Evaluation и улучшения стратегии Policy Improvement. После обучения была выполнена проверка качества полученной стратегии, а также сравнение с работой случайной стратегии.

2) Описание задания

В соответствии с заданием лабораторной работы необходимо реализовать алгоритм Policy Iteration для одной из сред обучения с подкреплением из библиотеки Gym или аналогичной библиотеки. При этом нельзя использовать среду Toy Text / Frozen Lake, рассмотренную в лекционном примере.

В данной работе для реализации алгоритма была выбрана среда Taxi-v3 из библиотеки Gym. Эта среда имеет дискретное пространство состояний и дискретное пространство

действий, поэтому она хорошо подходит для применения алгоритма Policy Iteration. Количество состояний в среде равно 500, а количество возможных действий равно 6.

В ходе выполнения лабораторной работы необходимо:

- создать среду Taxi-v3 и изучить ее основные характеристики;
- определить пространство состояний, пространство действий и структуру переходов;
- реализовать начальную стратегию агента;
- реализовать этап оценивания стратегии Policy Evaluation;
- реализовать этап улучшения стратегии Policy Improvement;
- выполнить полный алгоритм Policy Iteration до стабилизации стратегии;
- проверить работу обученного агента на нескольких тестовых эпизодах;
- сравнить качество обученной стратегии со случайной стратегией;
- сформировать итоговую таблицу результатов.

3) Текст программы и экранные формы с примерами выполнения

В данном разделе приведены основные этапы программной реализации алгоритма Policy Iteration для среды Taxi-v3. Все вычисления были выполнены в среде Google Colab с использованием языка программирования Python.

Для работы с задачей обучения с подкреплением использовалась библиотека Gym, которая предоставляет готовые среды для моделирования взаимодействия агента со средой. Для численных вычислений была подключена библиотека numpy, для формирования таблиц результатов — pandas, для построения графиков — matplotlib. Также использовались модуль time для измерения времени выполнения алгоритма и функция pprint для более удобного вывода структуры переходов среды.

🔴 # Импорт необходимых библиотек для выполнения лабораторной работы

```
try:
    import gym
except ImportError:
    !pip install gym==0.26.2
    import gym

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from pprint import pprint
import time

print("Библиотеки успешно импортированы")
print("Версия Gym:", gym.__version__)
print("Версия NumPy:", np.__version__)
```

```
... Gym has been unmaintained since 2022 and does not support NumPy 2.0 amongst other critical functionality.
Please upgrade to Gymnasium, the maintained drop-in replacement of Gym, or contact the authors of your software and request that they upgrade.
See the migration guide at https://gymnasium.farama.org/introduction/migration\_guide/ for additional information.
Библиотеки успешно импортированы
Версия Gym: 0.25.2
Версия NumPy: 2.0.2
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
return datetime.datetime.utcnow().replace(tzinfo=utc)
```

Рисунок 1 — Импорт необходимых библиотек

3.1. Создание среды Taxi-v3 и вывод ее характеристик

На первом этапе была создана среда Taxi-v3 из библиотеки Gym. Данная среда моделирует задачу управления такси, в которой агент должен забрать пассажира из одной точки и доставить его в пункт назначения. Среда имеет дискретное пространство состояний и дискретное пространство действий, что позволяет использовать алгоритм Policy Iteration без дополнительной дискретизации.

После создания среды были выведены ее основные характеристики. Пространство состояний содержит 500 возможных состояний, а пространство действий содержит 6 возможных действий. Также был выведен диапазон наград и начальное состояние среды. Полученная информация подтверждает, что выбранная среда подходит для реализации алгоритма Policy Iteration.

▶ # Создание среды Taxi-v3 и вывод основных характеристик среды

```
env = gym.make("Taxi-v3")

print("Среда:", "Taxi-v3")
print("Пространство состояний:")
pprint(env.observation_space)

print("\nПространство действий:")
pprint(env.action_space)

print("\nДиапазон наград:")
pprint(env.reward_range)

print("\nКоличество состояний:", env.observation_space.n)
print("Количество действий:", env.action_space.n)

state = env.reset()
if isinstance(state, tuple):
    state = state[0]

print("\nНачальное состояние:", state)
print("\nПример отображения среды:")
env.render()
```

Рисунок 2 — Код создания среды Taxi-v3

```
Среда: Taxi-v3
Пространство состояний:
Discrete(500)

Пространство действий:
Discrete(6)

Диапазон наград:
(-inf, inf)

Количество состояний: 500
Количество действий: 6

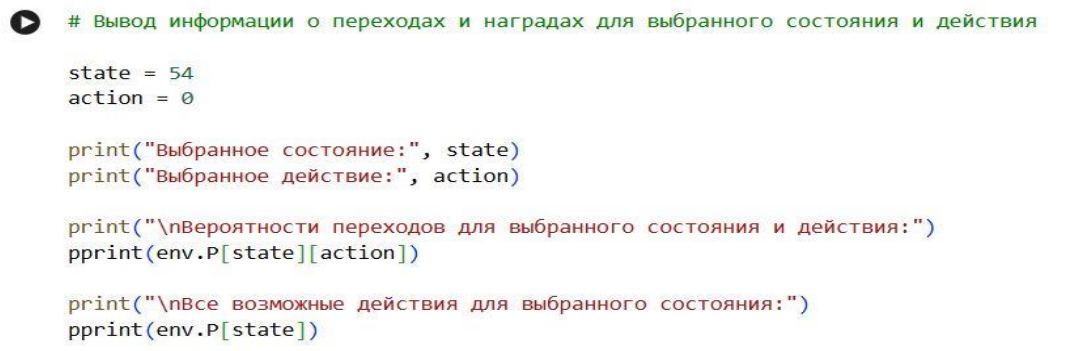
Начальное состояние: 54
```

Рисунок 3 — Результат создания среды Taxi-v3 и вывод характеристик

3.2. Анализ структуры переходов и наград среды

После вывода общих характеристик среды была рассмотрена структура переходов для выбранного состояния и действия. Для анализа было выбрано состояние 54 и действие 0. В среде Taxi-v3 для каждой пары «состояние–действие» задается список возможных переходов, содержащий вероятность перехода, следующее состояние, значение награды и признак терминального состояния.

Полученный результат показывает, что для выбранного состояния и действия переход является детерминированным: вероятность перехода равна 1.0. При выполнении действия 0 агент переходит из состояния 54 в состояние 154 и получает награду -1. Также были выведены все возможные действия для состояния 54. Для некоторых действий агент остается в том же состоянии и получает штраф -10, что соответствует некорректным действиям, например попытке посадки или высадки пассажира в неподходящем месте.



```
# Вывод информации о переходах и наградах для выбранного состояния и действия

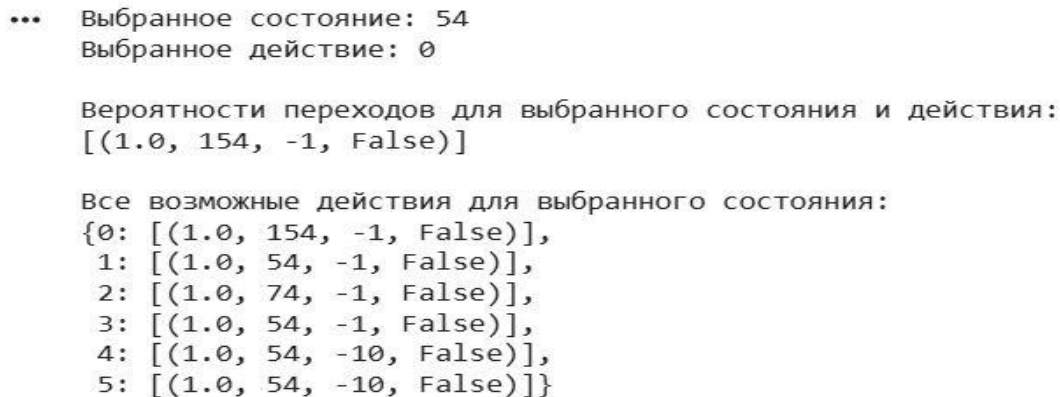
state = 54
action = 0

print("Выбранное состояние:", state)
print("Выбранное действие:", action)

print("\nВероятности переходов для выбранного состояния и действия:")
pprint(env.P[state][action])

print("\nВсе возможные действия для выбранного состояния:")
pprint(env.P[state])
```

Рисунок 4 — Код вывода вероятностей переходов среды Taxi-v3



```
... Выбранное состояние: 54
    Выбранное действие: 0

Вероятности переходов для выбранного состояния и действия:
[(1.0, 154, -1, False)]

Все возможные действия для выбранного состояния:
{0: [(1.0, 154, -1, False)],
 1: [(1.0, 54, -1, False)],
 2: [(1.0, 74, -1, False)],
 3: [(1.0, 54, -1, False)],
 4: [(1.0, 54, -10, False)],
 5: [(1.0, 54, -10, False)]}
```

Рисунок 5 — Результат вывода вероятностей переходов среды Taxi-v3

3.3. Создание агента и инициализация начальной стратегии

На следующем этапе был создан класс агента `PolicyIterationAgent`, реализующего основные элементы алгоритма `Policy Iteration`. При инициализации агента были определены количество состояний среды, количество возможных действий, массив доступных действий, начальная стратегия и начальные значения функции ценности состояний.

Начальная стратегия была задана равномерной: для каждого состояния вероятность выбора каждого из 6 действий равна $1/6$. Такой подход означает, что до начала обучения агент не имеет предпочтений между действиями. Вектор значений состояний был инициализирован нулями. Также были заданы параметры алгоритма: коэффициент дисконтирования $\gamma = 0.99$, порог сходимости $\theta = 10^{-8}$ и максимальное количество итераций 1000.

После создания агента были выведены его основные параметры и фрагмент начальной стратегии для первых состояний. Полученный результат подтверждает корректную инициализацию матрицы стратегии размером 500×6 и вектора ценностей размером 500.

▶ # Создание класса агента и инициализация начальной стратегии

```
class PolicyIterationAgent:
    """
    Класс агента, реализующего алгоритм Policy Iteration
    для дискретной среды Taxi-v3.
    """
    def __init__(self, env, gamma=0.99, theta=1e-8, max_iterations=1000):
        self.env = env

        # Количество состояний и действий
        self.observation_dim = env.observation_space.n
        self.action_dim = env.action_space.n

        # Массив доступных действий
        self.actions_variants = np.arange(self.action_dim)

        # Начальная стратегия: равномерный выбор каждого действия
        self.policy_probs = np.full(
            (self.observation_dim, self.action_dim),
            1 / self.action_dim
        )

        # Начальные значения функции ценности состояний
        self.state_values = np.zeros(self.observation_dim)

        # Параметры алгоритма
        self.gamma = gamma
        self.theta = theta
        self.max_iterations = max_iterations

        # История изменения функции ценности
        self.evaluation_history = []
        self.policy_changes_history = []

    def print_info(self):
        """
        Вывод основной информации об агенте.
        """
        print("Количество состояний:", self.observation_dim)
        print("Количество действий:", self.action_dim)
        print("Коэффициент дисконтирования gamma:", self.gamma)
        print("Порог сходимости theta:", self.theta)
        print("Максимальное количество итераций:", self.max_iterations)
        print("Размер матрицы стратегии:", self.policy_probs.shape)
        print("Размер вектора ценностей:", self.state_values.shape)

    def print_policy_fragment(self, count=10):
        """
        Вывод фрагмента начальной стратегии.
        """
        print("Фрагмент стратегии для первых", count, "состояний:")
        pprint(self.policy_probs[:count])

agent = PolicyIterationAgent(env)

agent.print_info()
print()
agent.print_policy_fragment(10)
```

Рисунок 6 — Код создания агента Policy Iteration


```
Количество состояний: 500
Количество действий: 6
... Коэффициент дисконтирования gamma: 0.99
Порог сходимости theta: 1e-08
Максимальное количество итераций: 1000
Размер матрицы стратегии: (500, 6)
Размер вектора ценностей: (500,)

Фрагмент стратегии для первых 10 состояний:
array([[0.16666667, 0.16666667, 0.16666667, 0.16666667, 0.16666667,
        0.16666667],
       [0.16666667, 0.16666667, 0.16666667, 0.16666667, 0.16666667,
        0.16666667],
       [0.16666667, 0.16666667, 0.16666667, 0.16666667, 0.16666667,
        0.16666667],
       [0.16666667, 0.16666667, 0.16666667, 0.16666667, 0.16666667,
        0.16666667],
       [0.16666667, 0.16666667, 0.16666667, 0.16666667, 0.16666667,
        0.16666667],
       [0.16666667, 0.16666667, 0.16666667, 0.16666667, 0.16666667,
        0.16666667],
       [0.16666667, 0.16666667, 0.16666667, 0.16666667, 0.16666667,
        0.16666667],
       [0.16666667, 0.16666667, 0.16666667, 0.16666667, 0.16666667,
        0.16666667],
       [0.16666667, 0.16666667, 0.16666667, 0.16666667, 0.16666667,
        0.16666667],
       [0.16666667, 0.16666667, 0.16666667, 0.16666667, 0.16666667,
        0.16666667]])
```

Рисунок 7 — Результат инициализации агента Policy Iteration

3.4. Реализация этапа оценивания стратегии Policy Evaluation

После инициализации агента был реализован этап оценивания стратегии Policy Evaluation. На данном этапе для текущей стратегии вычисляется функция ценности состояний $V(s)$. Она показывает ожидаемое суммарное дисконтированное вознаграждение, которое может получить агент, начиная из состояния s и далее следуя текущей стратегии.

В реализации для каждого состояния перебираются все возможные действия и все переходы, заданные моделью среды. Для каждого перехода учитываются вероятность перехода, получаемая награда, следующее состояние и коэффициент дисконтирования γ . Итерационный процесс продолжается до тех пор, пока максимальное изменение

функции ценности между соседними итерациями не станет меньше заданного порога θ , либо пока не будет достигнуто максимальное число итераций.

В результате оценивание начальной равномерной стратегии было выполнено в пределах заданного максимального числа итераций. Значения функции ценности для первых состояний оказались отрицательными, что является ожидаемым результатом для начальной случайной стратегии в среде Taxi-v3, так как агент часто выполняет неэффективные действия и получает штрафы.

```
# Реализация этапа оценивания стратегии Policy Evaluation

class PolicyIterationAgent:
    """
    Класс агента, реализующего алгоритм Policy Iteration
    для дискретной среды Taxi-v3.
    """
    def __init__(self, env, gamma=0.99, theta=1e-8, max_iterations=1000):
        self.env = env

        self.observation_dim = env.observation_space.n
        self.action_dim = env.action_space.n

        self.actions_variants = np.arange(self.action_dim)

        self.policy_probs = np.full(
            (self.observation_dim, self.action_dim),
            1 / self.action_dim
        )

        self.state_values = np.zeros(self.observation_dim)

        self.gamma = gamma
        self.theta = theta
        self.max_iterations = max_iterations

        self.evaluation_history = []
        self.policy_changes_history = []

    def print_info(self):
        """
        Вывод основной информации об агенте.
        """
        print("Количество состояний:", self.observation_dim)
        print("Количество действий:", self.action_dim)
        print("Коэффициент дисконтирования gamma:", self.gamma)
        print("Порог сходимости theta:", self.theta)
        print("Максимальное количество итераций:", self.max_iterations)
        print("Размер матрицы стратегии:", self.policy_probs.shape)
        print("Размер вектора ценностей:", self.state_values.shape)

    def print_policy_fragment(self, count=10):
```

```

def print_policy_fragment(self, count=10):
    """
    Вывод фрагмента стратегии.
    """
    print("Фрагмент стратегии для первых", count, "состояний:")
    pprint(self.policy_probs[:count])

def policy_evaluation(self):
    """
    Оценивание текущей стратегии.
    """
    self.evaluation_history = []

    for iteration in range(self.max_iterations):
        new_state_values = np.zeros(self.observation_dim)

        for state in range(self.observation_dim):
            state_value = 0

            for action in range(self.action_dim):
                action_probability = self.policy_probs[state][action]
                transition_value = 0

                for probability, next_state, reward, is_terminal in self.env.P[state][action]:
                    transition_value += probability * (
                        reward + self.gamma * self.state_values[next_state]
                    )

                state_value += action_probability * transition_value

            new_state_values[state] = state_value

        difference = np.max(np.abs(new_state_values - self.state_values))
        self.evaluation_history.append(difference)
        self.state_values = new_state_values

        if difference < self.theta:
            print("Оценивание стратегии завершено за", iteration + 1, "итераций")
            print("Максимальное изменение функции ценности:", difference)
            break

    return self.state_values

agent = PolicyIterationAgent(env)
state_values = agent.policy_evaluation()

print("\nФрагмент функции ценности для первых 20 состояний:")
print(state_values[:20])

```

Рисунок 8 — Код реализации этапа Policy Evaluation

...

Фрагмент функции ценности для первых 20 состояний:

```
[-364.93138404 -377.77869537 -375.36017601 -377.98693597 -395.64888943  
-393.28309066 -395.71344445 -395.21780063 -389.91514023 -390.85080962  
-384.16428272 -390.90760485 -396.17405944 -395.77070384 -396.22107692  
-393.84974505 -354.66893319 -374.60971144 -370.85583507 -374.93293113]
```

Рисунок 9 — Результат оценивания начальной стратегии

3.5. Реализация этапа улучшения стратегии Policy Improvement

После оценивания начальной стратегии был реализован этап улучшения стратегии Policy Improvement. На данном этапе для каждого состояния вычисляются значения $Q(s, a)$ для всех возможных действий. Значение $Q(s, a)$ показывает ожидаемое суммарное дисконтированное вознаграждение при выборе действия a в состоянии s с дальнейшим следованием текущей функции ценности.

Для каждого состояния выбирается действие с максимальным значением $Q(s, a)$. Если несколько действий имеют одинаковое максимальное значение, вероятность выбора распределяется между ними равномерно. Таким образом формируется новая улучшенная стратегия, которая является более эффективной по сравнению с начальной равномерной стратегией.

В результате выполнения данного этапа было изменено 3000 элементов матрицы стратегии. Это объясняется тем, что начальная стратегия была равномерной для всех 500 состояний и 6 действий, а после улучшения стратегия стала преимущественно детерминированной. Также были выведены фрагмент улучшенной стратегии и фрагмент Q -значений для первых состояний.

```

▶ # Реализация этапа улучшения стратегии Policy Improvement

def policy_improvement_for_agent(agent):
    """
    Улучшение стратегии на основе текущей функции ценности состояний.
    """
    q_values = np.zeros((agent.observation_dim, agent.action_dim))
    improved_policy = np.zeros((agent.observation_dim, agent.action_dim))

    for state in range(agent.observation_dim):
        for action in range(agent.action_dim):
            for probability, next_state, reward, is_terminal in agent.env.P[state][action]:
                q_values[state, action] += probability * (
                    reward + agent.gamma * agent.state_values[next_state]
                )

        best_action_value = np.max(q_values[state])
        best_actions = np.where(q_values[state] == best_action_value)[0]

        for best_action in best_actions:
            improved_policy[state, best_action] = 1 / len(best_actions)

    policy_changes = np.sum(agent.policy_probs != improved_policy)
    agent.policy_probs = improved_policy
    agent.policy_changes_history.append(policy_changes)

    print("Количество измененных элементов стратегии:", policy_changes)

    return improved_policy, q_values

improved_policy, q_values = policy_improvement_for_agent(agent)

print("\nФрагмент улучшенной стратегии для первых 20 состояний:")
print(improved_policy[:20])

print("\nФрагмент Q-значений для первых 10 состояний:")
print(q_values[:10])

```

Рисунок 10 — Код реализации этапа Policy Improvement

... Количество измененных элементов стратегии: 3000

Фрагмент улучшенной стратегии для первых 20 состояний:

```
[ [0. 0. 0. 0. 1. 0. ]  
  [0. 0. 0. 0. 1. 0. ]  
  [0. 0. 0. 0. 1. 0. ]  
  [0. 0. 0. 0. 1. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [1. 0. 0. 0. 0. 0. ]  
  [0. 0. 0. 0. 0. 1. ]  
  [0. 0.5 0. 0.5 0. 0. ]  
  [0. 0.5 0. 0.5 0. 0. ]  
  [0. 0.5 0. 0.5 0. 0. ]]
```

Фрагмент Q-значений для первых 10 состояний:

```
[ [-372.27418989 -362.2820702 -369.34666215 -362.2820702 -352.12224386  
  -371.2820702 ]  
  [-381.33067801 -375.00090842 -379.47615915 -375.00090842 -371.86361432  
  -384.00090842 ]  
  [-379.62578743 -372.60657425 -377.56927282 -372.60657425 -368.14727671  
  -381.60657425 ]  
  [-381.47747259 -375.20706661 -379.64034631 -375.20706661 -372.18360181  
  -384.20706661 ]  
  [-392.51649414 -392.69240054 -392.60824453 -392.69240054 -401.69240054  
  -401.69240054 ]  
  [-390.07833734 -390.35025975 -390.22016838 -390.35025975 -399.35025975  
  -399.35025975 ]  
  [-392.58302398 -392.75631 -392.67340762 -392.75631 -401.75631  
  -401.75631 ]  
  [-392.0722197 -392.26562262 -392.17309604 -392.26562262 -401.26562262  
  -401.26562262 ]  
  [-385.98385863 -387.01598883 -387.44403283 -387.01598883 -396.01598883  
  -396.01598883 ]  
  [-387.00609458 -387.94230153 -388.33056428 -387.94230153 -396.94230153  
  -396.94230153 ]]
```

Рисунок 11 — Результат улучшения начальной стратегии

3.6. Реализация полного алгоритма Policy Iteration

После реализации отдельных этапов оценивания и улучшения стратегии был реализован полный алгоритм Policy Iteration. Данный алгоритм последовательно выполняет два основных этапа: сначала оценивается текущая стратегия, затем на основе полученной функции ценности выполняется улучшение стратегии.

На каждой итерации сохраняется предыдущая стратегия, после чего выполняются Policy Evaluation и Policy Improvement. Затем новая стратегия сравнивается с предыдущей. Если стратегия больше не изменяется, алгоритм считается сошедшимся, так как дальнейшие итерации не приводят к изменению поведения агента.

В результате выполнения алгоритма Policy Iteration стратегия стабилизировалась за 6 итераций. Время выполнения алгоритма составило около 25.9 секунды. После завершения были выведены фрагмент итоговой стратегии и фрагмент итоговой функции ценности для первых состояний. Полученный результат показывает, что алгоритм успешно нашел устойчивую стратегию для среды Taxi-v3.



Реализация полного алгоритма Policy Iteration

```
def run_policy_iteration(agent):
    ...
    Полная реализация алгоритма Policy Iteration.
    На каждой итерации выполняется оценивание и улучшение стратегии.
    ...
    start_time = time.time()

    for iteration in range(agent.max_iterations):
        old_policy = agent.policy_probs.copy()

        agent.policy_evaluation()
        improved_policy, q_values = policy_improvement_for_agent(agent)

        policy_is_stable = np.array_equal(old_policy, improved_policy)

        print("Итерация:", iteration + 1)
        print("Стабильность стратегии:", policy_is_stable)
        print("-" * 50)

        if policy_is_stable:
            execution_time = time.time() - start_time
            print("Алгоритм Policy Iteration завершен")
            print("Количество итераций:", iteration + 1)
            print("Время выполнения:", execution_time)
            return iteration + 1, execution_time, q_values

    execution_time = time.time() - start_time
    print("Достигнуто максимальное количество итераций")
    print("Время выполнения:", execution_time)
    return agent.max_iterations, execution_time, q_values

agent = PolicyIterationAgent(env)
iterations_count, execution_time, final_q_values = run_policy_iteration(agent)

print("\nФрагмент итоговой стратегии для первых 20 состояний:")
print(agent.policy_probs[:20])

print("\nФрагмент итоговой функции ценности для первых 20 состояний:")
print(agent.state_values[:20])
```

Рисунок 12 — Код полного алгоритма Policy Iteration


```

... Итерация: 1
Стабильность стратегии: False
-----
Количество измененных элементов стратегии: 1342
Итерация: 2
Стабильность стратегии: False
-----
Количество измененных элементов стратегии: 1229
Итерация: 3
Стабильность стратегии: False
-----
Количество измененных элементов стратегии: 306
Итерация: 4
Стабильность стратегии: False
-----
Оценивание стратегии завершено за 12 итераций
Максимальное изменение функции ценности: 1.7053025658242404e-12
Количество измененных элементов стратегии: 18
Итерация: 5
Стабильность стратегии: False
-----
Оценивание стратегии завершено за 1 итераций
Максимальное изменение функции ценности: 1.7053025658242404e-12
Количество измененных элементов стратегии: 0
Итерация: 6
Стабильность стратегии: True
-----
Алгоритм Policy Iteration завершен
Количество итераций: 6
Время выполнения: 25.898879051208496

Фрагмент итоговой стратегии для первых 20 состояний:
[[0.  0.  0.  0.  1.  0. ]
 [0.  0.  0.  0.  1.  0. ]
 [0.  0.  0.  0.  1.  0. ]
 [0.  0.  0.  0.  1.  0. ]
 [0.5 0.  0.5 0.  0.  0. ]
 [0.5 0.  0.5 0.  0.  0. ]
 [0.5 0.  0.5 0.  0.  0. ]
 [0.5 0.  0.5 0.  0.  0. ]
 [1.  0.  0.  0.  0.  0. ]
 [1.  0.  0.  0.  0.  0. ]
 [1.  0.  0.  0.  0.  0. ]
 [1.  0.  0.  0.  0.  0. ]
 [0.5 0.  0.5 0.  0.  0. ]
 [0.5 0.  0.5 0.  0.  0. ]
 [0.5 0.  0.5 0.  0.  0. ]
 [0.5 0.  0.5 0.  0.  0. ]
 [0.  0.  0.  0.  0.  1. ]
 [0.5 0.  0.5 0.  0.  0. ]
 [1.  0.  0.  0.  0.  0. ]
 [0.5 0.  0.5 0.  0.  0. ]]

Фрагмент итоговой функции ценности для первых 20 состояний:
[944.72361809 864.01317574 903.55733909 873.75068256 789.53804327
 864.01317574 789.53804327 816.7669381 864.01317574 826.0272102
 903.55733909 835.3810204 807.59926872 826.0272102 807.59926872
 873.75068256 955.27638191 873.75068256 913.69428191 883.58654804]

```

Рисунок 13 — Результат выполнения алгоритма Policy Iteration

3.7. Формирование итоговой детерминированной стратегии

После завершения алгоритма Policy Iteration была сформирована итоговая детерминированная стратегия. Для этого для каждого состояния выбиралось действие с максимальной вероятностью в итоговой матрице стратегии. Так как после сходимости алгоритма стратегия стала в основном детерминированной, для большинства состояний выбирается одно конкретное действие.

Для удобства анализа были заданы текстовые обозначения действий среды Taxi-v3: South, North, East, West, Pickup и Dropoff. После этого был выведен фрагмент итоговой стратегии для первых 30 состояний. В результате для каждого состояния отображается номер выбранного действия и его текстовое название.

Данный этап позволяет интерпретировать найденную стратегию не только как числовую матрицу, но и как набор конкретных действий агента в различных состояниях среды.

```
▶ # Формирование итоговой детерминированной стратегии и расшифровка действий

action_names = {
    0: "South",
    1: "North",
    2: "East",
    3: "West",
    4: "Pickup",
    5: "Dropoff"
}

deterministic_policy = np.argmax(agent.policy_probs, axis=1)

print("Фрагмент итоговой детерминированной стратегии для первых 30 состояний:")
for state in range(30):
    action = deterministic_policy[state]
    print(
        "Состояние:",
        state,
        "| Действие:",
        action,
        "| Название действия:",
        action_names[action]
    )
```

Рисунок 14 — Код формирования итоговой детерминированной стратегии

Фрагмент итоговой детерминированной стратегии для первых 30 состояний:

Состояние: 0	Действие: 4	Название действия: Pickup
Состояние: 1	Действие: 4	Название действия: Pickup
Состояние: 2	Действие: 4	Название действия: Pickup
Состояние: 3	Действие: 4	Название действия: Pickup
Состояние: 4	Действие: 0	Название действия: South
Состояние: 5	Действие: 0	Название действия: South
Состояние: 6	Действие: 0	Название действия: South
Состояние: 7	Действие: 0	Название действия: South
Состояние: 8	Действие: 0	Название действия: South
Состояние: 9	Действие: 0	Название действия: South
Состояние: 10	Действие: 0	Название действия: South
Состояние: 11	Действие: 0	Название действия: South
Состояние: 12	Действие: 0	Название действия: South
Состояние: 13	Действие: 0	Название действия: South
Состояние: 14	Действие: 0	Название действия: South
Состояние: 15	Действие: 0	Название действия: South
Состояние: 16	Действие: 5	Название действия: Dropoff
Состояние: 17	Действие: 0	Название действия: South
Состояние: 18	Действие: 0	Название действия: South
Состояние: 19	Действие: 0	Название действия: South
Состояние: 20	Действие: 3	Название действия: West
Состояние: 21	Действие: 3	Название действия: West
Состояние: 22	Действие: 3	Название действия: West
Состояние: 23	Действие: 3	Название действия: West
Состояние: 24	Действие: 0	Название действия: South
Состояние: 25	Действие: 0	Название действия: South
Состояние: 26	Действие: 0	Название действия: South
Состояние: 27	Действие: 0	Название действия: South
Состояние: 28	Действие: 0	Название действия: South
Состояние: 29	Действие: 0	Название действия: South

Рисунок 15 — Фрагмент итоговой детерминированной стратегии

3.8. Проверка работы обученного агента

После формирования итоговой детерминированной стратегии была выполнена проверка работы обученного агента в среде Taxi-v3. Для этого агент запускался в нескольких тестовых эпизодах, в каждом из которых он выбирал действия в соответствии с найденной стратегией Policy Iteration.

Для каждого эпизода вычислялись суммарная награда и количество шагов до завершения. Также подсчитывалось количество успешных эпизодов. В качестве ограничения было задано максимальное число шагов в одном эпизоде, равное 200.

По результатам проверки обученный агент успешно завершил все 20 тестовых эпизодов. Средняя награда составила 7.95, а среднее количество шагов — 13.05. Полученные результаты показывают, что найденная стратегия позволяет агенту эффективно выполнять задачу перевозки пассажира в среде Taxi-v3.

```
▶ # Проверка работы обученного агента в среде Taxi-v3

def evaluate_trained_agent(env, policy, episodes_count=20, max_steps=200):
    """
    Проверка качества обученной стратегии на нескольких эпизодах.
    """
    rewards = []
    steps_list = []
    successful_episodes = 0

    for episode in range(episodes_count):
        reset_result = env.reset()
        state = reset_result[0] if isinstance(reset_result, tuple) else reset_result

        total_reward = 0
        steps = 0
        done = False
        truncated = False

        while not (done or truncated) and steps < max_steps:
            action = policy[state]
            step_result = env.step(action)

            if len(step_result) == 5:
                next_state, reward, done, truncated, info = step_result
            else:
                next_state, reward, done, info = step_result
                truncated = False

            total_reward += reward
            state = next_state
            steps += 1

            if total_reward > 0:
                successful_episodes += 1

        rewards.append(total_reward)
        steps_list.append(steps)

    return rewards, steps_list, successful_episodes

rewards, steps_list, successful_episodes = evaluate_trained_agent(
    env,
    deterministic_policy,
    episodes_count=20
)

print("Количество тестовых эпизодов:", len(rewards))
print("Количество успешных эпизодов:", successful_episodes)
print("Средняя награда:", np.mean(rewards))
print("Минимальная награда:", np.min(rewards))
print("Максимальная награда:", np.max(rewards))
print("Среднее количество шагов:", np.mean(steps_list))

print("\nНаграды по эпизодам:")
print(rewards)

print("\nКоличество шагов по эпизодам:")
print(steps_list)
```

Рисунок 16 — Код проверки обученного агента

```
... Количество тестовых эпизодов: 20
Количество успешных эпизодов: 20
Средняя награда: 7.95
Минимальная награда: 5
Максимальная награда: 14
Среднее количество шагов: 13.05

Награды по эпизодам:
[5, 11, 10, 5, 8, 14, 9, 5, 8, 6, 9, 12, 6, 5, 9, 7, 5, 8, 10, 7]

Количество шагов по эпизодам:
[16, 10, 11, 16, 13, 7, 12, 16, 13, 15, 12, 9, 15, 16, 12, 14, 16, 13, 11, 14]
```

Рисунок 17 — Результат проверки обученного агента

3.9. Сравнение обученной стратегии со случайной стратегией

Для оценки эффективности найденной стратегии было выполнено сравнение обученного агента со случайной стратегией. Случайный агент на каждом шаге выбирает одно из доступных действий без учета текущего состояния и без использования функции ценности.

Обе стратегии были протестированы на 20 эпизодах в одинаковой среде Taxi-v3. Для каждой стратегии были вычислены количество успешных эпизодов, средняя награда, минимальная и максимальная награда, а также среднее количество шагов.

Результаты сравнения показали существенное преимущество стратегии, полученной методом Policy Iteration. Случайная стратегия не завершила успешно ни одного эпизода из 20 и получила среднюю награду -786.35. В то же время стратегия Policy Iteration успешно завершила все 20 эпизодов, получила среднюю награду 7.95 и потребовала в среднем 13.05 шага. Это подтверждает эффективность реализованного алгоритма.

```

▶ # Сравнение обученной стратегии со случайной стратегией

def evaluate_random_agent(env, episodes_count=20, max_steps=200):
    """
    Проверка качества случайной стратегии на нескольких эпизодах.
    """
    rewards = []
    steps_list = []
    successful_episodes = 0

    for episode in range(episodes_count):
        reset_result = env.reset()
        state = reset_result[0] if isinstance(reset_result, tuple) else reset_result

        total_reward = 0
        steps = 0
        done = False
        truncated = False

        while not (done or truncated) and steps < max_steps:
            action = env.action_space.sample()
            step_result = env.step(action)

            if len(step_result) == 5:
                next_state, reward, done, truncated, info = step_result
            else:
                next_state, reward, done, info = step_result
                truncated = False

            total_reward += reward
            state = next_state
            steps += 1

            if total_reward > 0:
                successful_episodes += 1

            rewards.append(total_reward)
            steps_list.append(steps)

    return rewards, steps_list, successful_episodes

random_rewards, random_steps, random_successful = evaluate_random_agent(
    env,
    episodes_count=20
)

comparison_table = pd.DataFrame({
    "Стратегия": ["Случайная стратегия", "Стратегия Policy Iteration"],
    "Количество успешных эпизодов": [random_successful, successful_episodes],
    "Средняя награда": [np.mean(random_rewards), np.mean(rewards)],
    "Минимальная награда": [np.min(random_rewards), np.min(rewards)],
    "Максимальная награда": [np.max(random_rewards), np.max(rewards)],
    "Среднее количество шагов": [np.mean(random_steps), np.mean(steps_list)]
})

comparison_table

```

Рисунок 18 — Код сравнения обученной и случайной стратегий

	Стратегия	Количество успешных эпизодов	Средняя награда	Минимальная награда	Максимальная награда	Среднее количество шагов
0	Случайная стратегия	0	-786.35	-938	-668	200.00
1	Стратегия Policy Iteration	20	7.95	5	14	13.05

Рисунок 19 — Сравнение обученной и случайной стратегий

3.10. Анализ наград по эпизодам

Для наглядного сравнения качества работы случайной и обученной стратегий был построен график суммарных наград по тестовым эпизодам. По оси абсцисс отложен номер эпизода, а по оси ординат — суммарная награда, полученная агентом за эпизод.

Из графика видно, что случайная стратегия получает большие отрицательные значения награды во всех эпизодах. Это связано с тем, что случайный агент часто выполняет неправильные действия, получает штрафы и не может эффективно завершить задачу. В отличие от нее, стратегия Policy Iteration во всех эпизодах показывает положительные или близкие к положительным значения награды.

Таким образом, график подтверждает выводы, полученные при анализе сравнительной таблицы: стратегия, найденная алгоритмом Policy Iteration, значительно эффективнее случайной стратегии и обеспечивает устойчивое выполнение задачи Taxi-v3.

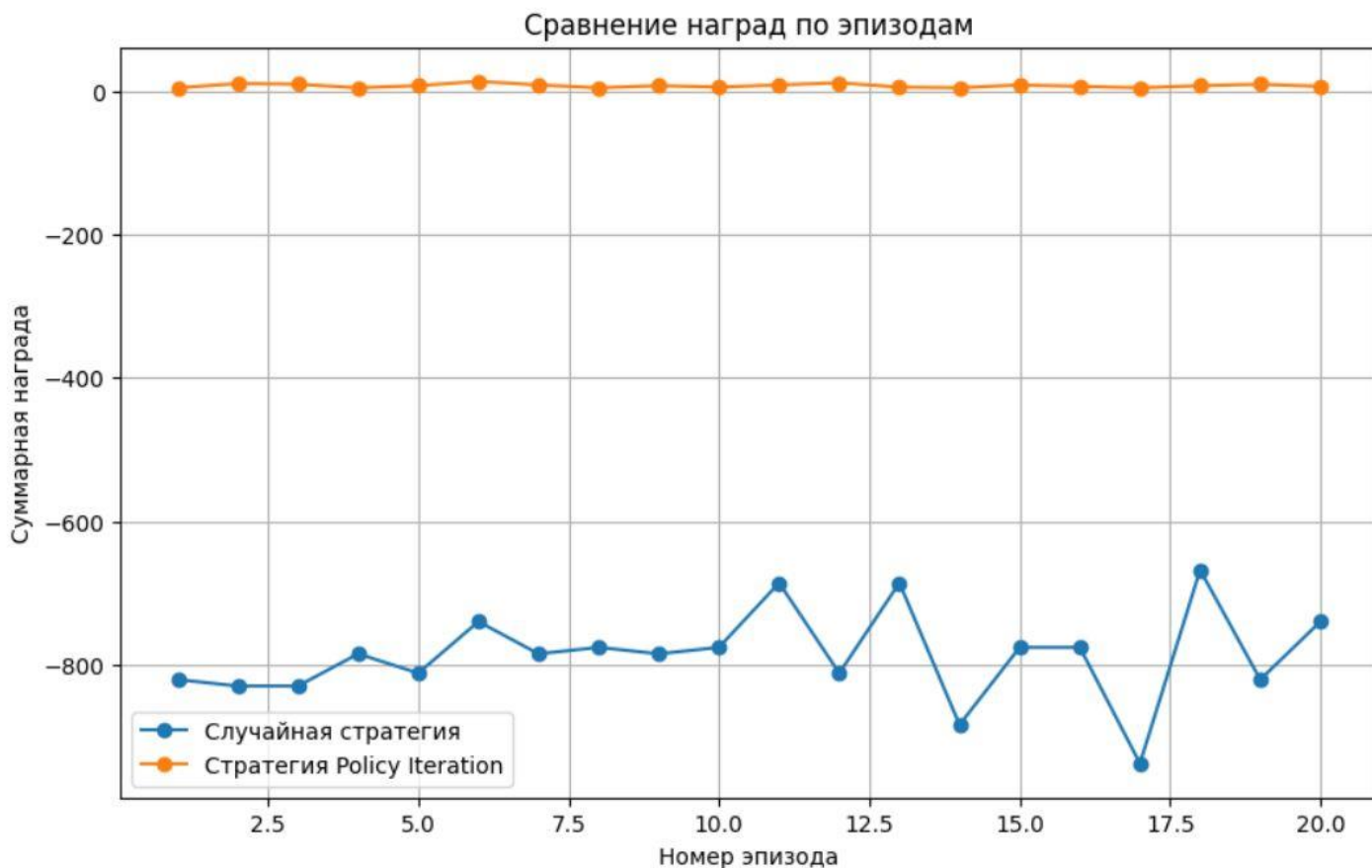


Рисунок 20 — График сравнения наград случайной и обученной стратегий

3.11. Формирование итоговой таблицы результатов

На заключительном этапе выполнения программы была сформирована итоговая таблица результатов лабораторной работы. В таблицу были включены основные характеристики среды и полученного решения: название среды, количество состояний, количество действий, коэффициент дисконтирования, порог сходимости, количество итераций алгоритма Policy Iteration, время выполнения, количество тестовых эпизодов, количество успешных эпизодов, средняя награда обученного агента и среднее количество шагов.

Итоговая таблица показывает, что алгоритм Policy Iteration был применен к среде Taxi-v3, содержащей 500 состояний и 6 действий. При значении коэффициента дисконтирования $\gamma = 0.99$ и пороге сходимости $\theta = 10^{-8}$ алгоритм завершился за 6 итераций. Обученная стратегия успешно прошла все 20 тестовых эпизодов, что подтверждает корректность реализации алгоритма и эффективность найденной политики.

```
 # Формирование итоговой таблицы результатов лабораторной работы

final_results = pd.DataFrame({
    "Показатель": [
        "Среда",
        "Количество состояний",
        "Количество действий",
        "Коэффициент дисконтирования",
        "Порог сходимости",
        "Количество итераций Policy Iteration",
        "Время выполнения алгоритма",
        "Количество тестовых эпизодов",
        "Количество успешных эпизодов",
        "Средняя награда обученного агента",
        "Среднее количество шагов обученного агента"
    ],
    "Значение": [
        "Taxi-v3",
        env.observation_space.n,
        env.action_space.n,
        agent.gamma,
        agent.theta,
        iterations_count,
        execution_time,
        len(rewards),
        successful_episodes,
        np.mean(rewards),
        np.mean(steps_list)
    ]
})

final_results
```

Рисунок 21 — Код формирования итоговой таблицы результатов лабораторной работы

...

	Показатель	Значение
0	Среда	Taxi-v3
1	Количество состояний	500
2	Количество действий	6
3	Коэффициент дисконтирования	0.99
4	Порог сходимости	10^{-8}
5	Количество итераций Policy Iteration	6
6	Время выполнения алгоритма	25.898879
7	Количество тестовых эпизодов	20
8	Количество успешных эпизодов	20
9	Средняя награда обученного агента	7.95
10	Среднее количество шагов обученного агента	13.05

Рисунок 22 — Итоговая таблица результатов лабораторной работы

4) Заключение

В ходе лабораторной работы был изучен и реализован алгоритм Policy Iteration для задачи обучения с подкреплением. В качестве среды была выбрана Taxi-v3 из библиотеки Gym, так как она имеет дискретное пространство состояний и действий, а также предоставляет модель переходов, необходимую для применения методов динамического программирования.

В процессе выполнения работы были рассмотрены основные характеристики среды: количество состояний, количество действий, структура переходов и система наград. После этого был создан агент, задана начальная равномерная стратегия и реализованы два основных этапа алгоритма: Policy Evaluation и Policy Improvement. Полный алгоритм Policy Iteration завершился за 6 итераций, после чего была получена стабильная детерминированная стратегия.

Для проверки качества найденной стратегии обученный агент был протестирован на 20 эпизодах. Агент успешно завершил все 20 эпизодов, средняя награда составила 7.95, а среднее количество шагов — 13.05. Дополнительно была выполнена проверка случайной стратегии. Сравнение показало, что случайная стратегия не завершила успешно ни одного эпизода и получила значительно худшую среднюю награду.

Таким образом, все поставленные задачи лабораторной работы были выполнены. Реализованный алгоритм Policy Iteration позволил найти эффективную стратегию поведения агента в среде Taxi-v3 и показал существенное преимущество по сравнению со случайным выбором действий.